



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

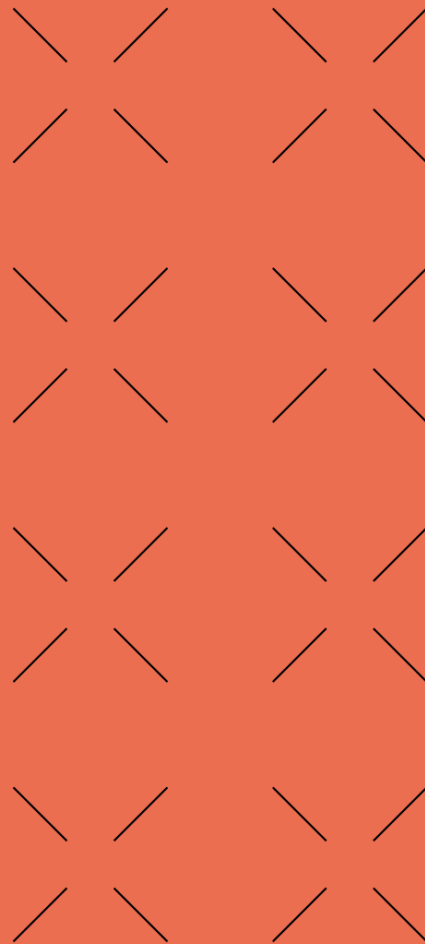
CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 03 — Variables, tipus i operadors

Bloc 02

1r CFGS DAW · Programació

Sergi García Barea



Scanner: llegir pel teclat

La idea en una frase

“ Scanner és la classe de Java que llegeix el que escrius pel teclat: instancies un objecte amb `new Scanner(System.in)` i li demanes dades amb `nextInt()`, `nextDouble()` o `nextLine()` .

1 Importar — `import java.util.Scanner;`
demanes el llibre a la biblioteca

2 Instanciar — `Scanner sc = new Scanner(System.in);` (el constructor, com `new String(...)`)

3 Demanar dades — mètodes `next...` :
cada un espera que escrigues i pulses Enter

4 Tancar — `sc.close();` en acabar

✓ A partir d'ací els teus programes tenen **oïdes**: conversors, calculadores, registres de notes... programes de veritat.

Demandar dades: els mètodes next

PrimerEscucha.java

Java

```
import java.util.Scanner;

public class PrimerEscucha {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("¿Cómo te llamas? ");
        String nombre = sc.nextLine();
        System.out.print("¿Cuántos años tienes? ");
        int edad = sc.nextInt();
        System.out.print("¿Nota media? ");
        double nota = sc.nextDouble();
        System.out.println("Hola, " + nombre
            + ". " + edad + " anys i un " + nota);
        sc.close();
    }
}
```

Els mètodes més usats

Mètode	Llig
<code>nextLine()</code>	Una línia de text completa
<code>next()</code>	Només la següent paraula
<code>nextInt()</code>	Un número enter
<code>nextDouble()</code>	Un número amb decimals
<code>nextBoolean()</code>	true o false



L'embolic de nextLine() després de nextInt()

L'error més odiat del Scanner

Trampa.java

Java

```
System.out.print("Edad: ");  
int edad = sc.nextInt(); // escrius 18 + Enter  
System.out.print("Nombre: ");  
String nombre = sc.nextLine();  
// ¡es salta la pregunta! retorna ""
```



Quan fas `nextInt()`, l'Enter que vas pulsar **queda al buffer**. El següent `nextLine()` se'l menja i retorna una línia buida.

La solució: el guardià del buffer

Arreglat.java

Java

```
int edad = sc.nextInt();  
sc.nextLine(); // es menja l'Enter sobrant  
String nombre = sc.nextLine(); // ara sí
```



Memoritza el reflex: **número** → **nextInt()** buit → **text**. Apareix en tots els exàmens.

Consola: eixida amb format

La idea en una frase

“ `printf` i `String.format` donen format a la teua eixida (decimals, amplària, alineació) en una línia, i conèixer els errors típics del Scanner t'estalvia els bugs més odiats de la unitat.

Format.java

Java

```
String nom = "Ana";  
int edat = 20;  
double nota = 9.5;  
System.out.printf("Nom: %s, Edat: %d, Nota: %.2f%n",  
    nom, edat, nota);  
// Nom: Ana, Edat: 20, Nota: 9,50
```

Especificadors bàsics

Especificador	Tipus
<code>%s</code>	String
<code>%d</code>	Enter
<code>%f</code>	Decimal (<code>%.2f</code> = 2 decimals)
<code>%c</code>	Caràcter
<code>%n</code>	Salt de línia (multiplataforma)

Decimals, amplària i String.format

Controlar com es veu

Amplaria.java

Java

```
double pi = Math.PI;
System.out.printf("%.2f%n", pi); // 3,14
System.out.printf("%10.2f%n", pi); // alineat dreta
System.out.printf("%-10.2f%n", pi); // alineat esquerra
```



Els decimals usen la **configuració regional**: locale espanyol escriu `3,14` (coma); anglés, `3.14` (punt). No t'espantes si varia.



String.format: el mateix sense imprimir

Missatge.java

Java

```
String msg = String.format(
    "Benvingut, %s. Tens %d missatges.",
    "Carles", 3);
// Benvingut, Carles. Tens 3 missatges.
```



`printf` per a **imprimir**.

`String.format` per a **guardar el text** format com a valor.

Errors clàssics del Scanner

I els seus remeis

- 1 **Oblidar l'import** — sense `import java.util.Scanner;`, Java no coneix la classe
- 2 **No tancar-lo** — `sc.close()` sempre en acabar
- 3 **Tipus equivocats** — demanes `int`, escriuen lletres → ✨ `InputMismatchException`
- 4 **Enter residual** — `nextLine()` després de `nextInt()` (recordatori)

El validador que no es trenca

EdatSegura.java

Java

```
int edat = -1;
while (edat == -1) {
    System.out.print("Quants anys tens? ");
    if (sc.hasNextInt()) {
        edat = sc.nextInt();
    } else {
        System.out.println("Això no és un enter.");
        sc.next(); // descarta la brossa
    }
}
```



`hasNextInt()` **no consumeix** la dada: si no és enter, consumix la brossa amb `sc.next()` abans de tornar a preguntar.

Math.random(): el casino de Java

La idea en una frase

“

`Math.random()` retorna un nombre aleatori entre **0.0 (inclòs) i 1.0 (exclòs)**, i amb la fórmula `(int)(Math.random() * (max - min + 1)) + min` el converteixes en un dau, una loteria o qualsevol nombre que necessites.

Casino.java

Java

```
double aleatorio = Math.random(); // 0.0-0.999...
int deCeroANueve = (int) (Math.random() * 10); // 0-9
int dado = (int) (Math.random() * 6) + 1; // 1-6
```

1

`Math.random() * 6` → entre 0.0 i 5.999...

2

`(int)` el trunca → entre 0 i 5

3

`+ 1` el desplaça → entre **1 i 6** ✓

i

És un mètode **estàtic**: es crida amb `Math.nombre`, sense crear objectes.

La fórmula universal i la classe Math

De 0.0–1.0 al que tu vulgues

La fórmula universal

Formula.java

Java

```
int numero = (int) (Math.random()
    * (max - min + 1)) + min;
// entre 5 i 10
int n = (int) (Math.random() * 6) + 5;
```

 **Trunca abans de sumar:** `(int)(... * 6) + 1`, mai `(int)(... * 6 + 1)`. Si sumes dins del casting, el rang canvia i els teus daus mentiran.

La sala de màquines de Math

Math.java

Java

```
Math.PI           // 3.14159...
Math.pow(2, 10)   // 1024.0
Math.sqrt(144)    // 12.0
Math.abs(-7)      // 7.0
Math.round(4.6)   // 5 (redonix)
Math.ceil(4.1)    // 5.0 (sostre)
Math.floor(4.9)   // 4.0 (sòl)
Math.max(3, 9)    // 9
Math.min(3, 9)    // 3
```

Exercici: el dau que menteix

Quin rang produïx cada línia?



Quina de les tres és la que dona un dau de veritat (1 a 6)?

Dauses.java

Java

```
int a = (int) (Math.random() * 6);  
int b = (int) (Math.random() * 6) + 1;  
int c = (int) (Math.random() * 7);
```

Solució

- **a:** 0.0–5.999 truncat → **0 a 5**
- **b:** l'anterior + 1 → **1 a 6**  el dau de veritat
- **c:** 0.0–6.999 truncat → **0 a 6** (set cares, i el 0 no existeix!)

 La diferència entre b i c és subtil però decisiva: la `+ 1` ha d'anar **fora** del casting.

Mètodes útils de String

La idea en una frase

“ String porta de sèrie una caixa de ferramentes per a **mesurar, retallar, buscar i transformar text**: `length()`, `trim()`, `toUpperCase()`, `substring()`, `replace()` ...

Gimnas.java

Java

```
String texto = " Programación DAW ";
texto.length();      // 20 (espais inclosos)
texto.trim();        // "Programación DAW"
texto.toUpperCase(); // MAJÚSCULES
texto.toLowerCase(); // minúscules
texto.contains("DAW"); // true
texto.indexOf("DAW"); // 15
texto.substring(2, 14); // "Programación"
texto.replace("DAW", "DAM");
```

Cerca.java

Java

```
String email = "ana@instituto.edu";
email.contains("@"); // true
email.endsWith(".edu"); // true
email.indexOf("@"); // 3 (-1 si no és)
String sucio = " Ana ";
sucio.trim(); // "Ana"
```



En `substring(inicio, fin)` el fin no s'inclou · `indexOf()` retorna -1 si no troba.

length() porta parèntesis i l'exemple guiat

Mètodes d'objecte: texto.metodo()

 length() és un **mètode** (amb parèntesis).
Escriure `texto.length` no compila. (Els arrays usen `.length` sense parèntesis — la U06.)

Exemple guiat: el nom de l'usuari

```
ProcesaNombre.java

String nombre = " aNA ";
String limpio = nombre.trim();           // "aNA"
String mayus = limpio.toUpperCase();     // "ANA"
String primera = mayus.substring(0, 1); // "A"
String ultima = mayus.substring(mayus.length() - 1); // "A"
```



Exercici: la inicial d'una reina. Sense executar, què imprimeix?

Reina.java

Java

```
String nombre = " merida ";
String inicial = nombre.trim().toUpperCase().substring(0, 1);
String resto = nombre.trim().substring(1).toLowerCase();
System.out.println(inicial + ". " + resto);
```



Solució: `M. erida` — trim → toUpperCase → substring(0,1) = "M"; trim → substring(1) = "erida".
(La idea era normalitzar "M. erida"... encara que el resultat sona a princesa amb pressa.)

Repàs: la caixa que no cabia

Sense executar: què imprimeix?



Eres la JVM. Acaben de donar-te este programa per a executar. Què imprimeixes?

Misterio.java

Java

```
int nota = 7;
int sobre = 10;
System.out.println("Nota " + nota + sobre);
System.out.println("Nota " + (nota + sobre));
System.out.println(nota / 2 + " de nota media");
```



Solució: Nota 710 · Nota 17 · 3 de nota media

- Quan un `+` mescla text i números, Java **concatena**: "Nota " + 7 + 10 → "Nota 710"
- Els parèntesis `(nota + sobre)` forcen la suma → "Nota 17"
- `7 / 2` és divisió entera: **3**, no 3.5



Tres trampes de la unitat en un sol programa: concatenació, parèntesis i divisió entera.

Fireside chat: int vs double

Dos caixes del magatzem discuteixen

i int: — Jo sóc la caixa de mudança. Compacta, exacta, sense decimals. 17 dividit entre 5 són 3 i s'ha acabat.

✓ double: — *alça una cella ¿3?* Per a mi són 3.4. Jo guarde els decimals de veritat: preus, notes mitjanes, temperatures...

i int: — Per a contar, bucles i edats, em criden a mi. Contar amb decimals no té sentit.

✓ double: — I mesurar i calcular mitjanes, em criden a mi. Saps quantes voltes he vist a novats escriure `(int)(Math.random() * 6 + 1)` i plorar perquè el dau mai no eixia 6?

i int: — *gruny* Un equip. Tu per al sencer, jo per al fi. I long per als astronòmics, char per a les lletres.

⚠ La lliçó: int per al sencer, double per al decimal. Confondre'ls en una divisió (o en un casting mal posat) produïx els errors més clàssics d'esta unitat.

Laboratori de tortura: el caixer que cobra malament

3 errors de compilació + 1 de lògica (30 min)

Tortura.java

Pista: 4 errors

```
import java.util.Scanner;
public class Tortura {
    public static void main(string[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Cantidad a retirar: ");
        int cantidad = sc.nextInt()
        double billetes5 = cantidad / 5.0;
        System.out.println("Te dan " + billetes5 + " billetes de 5");
        sc.close();
    }
}
```



Fes que compile, que execute i que **tota** l'eixida siga correcta. Prova amb `cantidad = 17`: quants bitllets de 5 haurien de ser?



Solució

- `string[]` → `String[]` (majúscula)
- Falta la `{` del cos del main
- Falta el `;` de `sc.nextInt()`
- **Lògica:** `cantidad / 5.0` dona 3.4 bitllets
— un caixer no dona 3.4 bitllets → `int`
`billetes5 = cantidad / 5;`

Atreveix-te a pensar

Preguntes finals i d'entrevista

- 1 Sense executar:** què imprimeix
`System.out.println("a/b = " + 10 / 3); ?` |
amb `(double) 10 / 3` ?
- 2 El preu que no quadra:** `double total = precio * 0.21;` mostra 21.0000000000000004. Com ho arregles amb ferramentes d'esta unitat?
- 3 Ternari encadenat:** assigna "niño" / "adulto" / "jubilado" segons l'edat
- 4 V o F:** "Math.random() * 5 pot retornar el número 5"

💡 Solucions

- `a/b = 3` (entera) · `a/b real = 3.333...`
(força decimal)
- Coma flotant binària: `Math.round(total * 100) / 100.0` o `printf "%.2f`
- `edad < 12 ? "niño" : (edad < 65 ? "adulto" : "jubilado")`
- **Fals:** `random()` mai arriba a 1 → (int)
(`random()*5`) dona 0-4

i Pregunta d'entrevista real: "explica'm, com si tinguera 8 anys, què és una variable i un tipus primitiu."

Resum — Bloc 02 completat

El que hem après



Scanner

Importar, instanciar, next... · l'Enter residual i hasNextInt() com a guardià



Eixida formatada

printf / String.format / NumberFormat · %d, %s, %.2f, %n



Math.random()

Fórmula universal $(\text{max} - \text{min} + 1) \cdot \text{Math.random()} + \text{min}$ · Math.round, pow, sqrt...



String + repàs

length(), trim(), substring(), indexOf() · concatenació, divisió entera, ternari encadenat



Ja saps guardar dades, llegir del teclat, calcular i donar format! A la propera unitat: presa de decisions amb if/else. 🚀

